EPL484 / SOFTWARE EVOLUTION / PROJECT 1

Guava
Software evolution and architecture recovery

A reproducible study of thirty sporadically selected stable Java releases, with a paper-based replacement for JNode and one frozen configuration for each recovery condition.

HISTORY	EXPERIMENT	BOUNDARY
30 stable releases 10.0 → 33.7.2	**120 final partitions** ACDC and KMeans × full/filtered	**Internal classes only** Merged inner/anonymous types
26 major/minor families	Exact lecture cohesion, coupling and MQ	Guava JRE production artifact

Study boundary and evidence

The observed graph grows from **317** to **591 canonical classes**, and from **1,693** to **3,969 directed internal dependencies**. These are measurements of a static implementation view. They are not a claim about Guava's intended architecture or its users' experience.

Release window: 2011-09-28–2026-09-29. Dataset snapshot: 2026-10-17. Repository: github.com/google/guava.

All measurements in this document are loaded from saved result files. Source code, raw inputs, exact assignments and a machine-readable audit trail accompany the report. Student names and contributions must be supplied truthfully by the submitting team.

1 Findings and experiment map

Structural scale increases across the sampled history. Canonical classes change by 86.4% and internal directed dependencies by 134.4% between the endpoints. Irregular sampling describes broad changes; it does not identify the exact intervening commit that caused them. [D1; L12 pp.35–48]

Usage Equality produces an explicit, inspectable exclusion set. The replacement detects 6–24 omnipresent classes per release; 31 distinct class identities are detected at least once. Filtering removes those nodes and their incident edges from the analysis graph. It does not delete code from Guava. [D1; D2]

ACDC, after holding the evaluated graph fixed: reclustering the retained graph increases MQ in 23 sampled releases, decreases it in 7, and leaves it unchanged within 10^{-12} in 0. This is a structural metric result, without an authoritative architecture against which to measure accuracy. [D3]

KMeans, after holding the evaluated graph fixed: reclustering the retained graph increases MQ in 26 sampled releases, decreases it in 4, and leaves it unchanged within 10^{-12} in 0. This is a structural metric result, without an authoritative architecture against which to measure accuracy. [D3]

Condition	Algorithm	Graph evaluated
A1	Original ACDC (+so, -l20)	All canonical internal classes
A2	Same ACDC configuration	Induced graph after UE removal
A3	Sparse normalized KMeans	All canonical internal classes
A4	Same features and version-derived k	Induced graph after UE removal

The assignment requires all four conditions. The additional note requesting one analysis per version is implemented as **one final partition per condition per release**, following a single-version pilot. KMeans' ten internal initializations produce one fitted result. The latest-release AI experiment is reported separately, as required by the assignment. [AS pp.2–4]

The recorded AI proposal groups the latest 591 classes into 18 named modules. Its full-graph MQ is 0.0987 versus 0.1006 for KMeans and 0.4032 for ACDC. The AI also sees names and package documentation, so this is not a controlled test of superiority from identical information. Its conceptual boundaries and confidence limits are discussed alongside the numerical result. [D12]

2 Subject and sporadic sampling

The students selected **google/guava**. The study follows the published main Guava JRE artifact, providing a consistent production boundary across a long release history. This is a library's implemented class architecture; other repository modules are outside the sampled artifact. [G1; D4]

Course eligibility criterion	Recorded evidence
Java; latest release ≥10,000 LOC	GitHub primary language: Java. 95,876 nonblank, noncomment lines in owned Java sources.
GitHub repository created before 2024	2014-05-29
Not a fork; not archived	fork=false; archived=false at the recorded snapshot.
≥50 commits since 2024-01-01	100 distinct qualifying commits retained as a lower-bound witness, rather than an inferred total.
≥10 major/minor releases	30 sampled stable releases covering 26 distinct major/minor families.

The eligibility SLOC count includes source-owned package descriptors. The narrower source-file population linked to canonical class nodes contains 95,800 code lines at the latest release. Both exceed the requirement; the denominators are declared rather than treated as interchangeable. [D4; D6]

Selection is specified before metric inspection

The captured tag catalog contains 124 refs, of which 82 match the numeric stable-release pattern. Forty-two nonmatching refs include prereleases, auxiliary artifacts and alternate tag names; they are not all beta releases. Nine numeric releases before 10.0 are outside the defined sampling window. The ambiguous 13.0 duplicate naming and two later backports to earlier lines are also excluded, leaving 70 chronological mainline candidates. [D4]

> Candidate index for sample i: **round(i × 69 / 29)**, i = 0,...,29

This endpoint-preserving rule selects 30 of the 70 eligible points and leaves 40 eligible points unsampled. Every neighboring sampled pair skips at least one actual stable tag; the exact intervening list is validated against the captured catalog. Early artifacts use plain versions, and the sample uses the JRE variant after Guava introduced the variant suffix. Android variants are not separate observations. [D4]

Eligibility evidence does not establish instructor approval. No teacher approval is represented as observed. The publication dates of old releases come from Guava's release-history pages; later dates use GitHub Release publication timestamps. Git commit dates remain separate fields. [AS p.1; G2; D4]

3 Data acquisition and provenance

Each selected release has an immutable Git commit identity and three Maven Central artifacts: the production binary JAR, its source JAR and the POM. Acquisition reads public data and never executes upstream Guava code. Published binaries avoid historical build failures becoming a hidden release-selection criterion. [D4; D5]

Evidence layer	Stored and checked
Release identity	Git tag name, 40-character commit SHA, published date, date basis and artifact coordinate.
Artifact integrity	Pinned SHA-256 for every downloaded file; Maven SHA-1 sidecars; ZIP CRC checks; POM version/artifact identity.
Source ownership	Per-source-file path and checksum, source-owned class inventory, SLOC definition and excluded binary entries.
Source-to-Git audit	Every Java source-JAR file at the oldest and newest selected endpoints compared byte for byte with the pinned Git archive.
Offline reproducibility	Cached artifacts must match existing pins; corrupt inputs are not silently overwritten.

Endpoint	Java files	Git/source result	Commit prefix
10.0	326	All byte-identical	baf008c5a3de
33.7.2	611	All byte-identical	666338cc49d5

All **90 artifacts** (122.02 MiB) passed pinned checks. Full Git-source correspondence was checked at the two endpoints; the intermediate 28 releases have artifact and Git identity pins but are not claimed to have the same complete per-file Git comparison. SHA sidecars fetched from the same repository provide integrity evidence, not independent authorship authentication. PGP signatures were not verified. [D5]

How to explain the data in the oral exam

Start at a row in `config/versions.json`. Show its tag SHA and Maven URL, hash the cached JAR, then inspect the source manifest and extractor provenance. Finally show how the canonical class and edge CSVs become the algorithm inputs. This chain makes an individual measurement traceable back to a concrete upstream artifact. [L4 pp.11–15]

Audit pointers: config/versions.json; data/provenance/SHA256SUMS; acquisition.json; source_inventory_<version>.json; git_source_match_10.0.json; git_source_match_33.7.2.json. All paths are repository-relative.

4 The structural graph

The extractor reads class files using ASM. A node is a source-owned outermost type in `com.google.common`. Genuine member, local and anonymous classes are merged with their enclosing class using class-file ownership metadata. Interfaces, enums, annotations and records follow the same node rule. A dollar sign alone is not proof of nesting. [L5 PDF pp.22–23; D6]

Relation	Direction	Evidence
EXTENDS	Subtype → parent type	Declared superclass or parent interface
IMPLEMENTS	Class → interface	Declared implementation relationship
METHOD_CALL	Caller → target owner	Method invocation, with declaring-member resolution
METHOD_PARAMETER	User → parameter type	Reference types in declared/invoked method descriptors
FIELD_REFERENCE	User → field owner	Field access, with declaring-member resolution
TYPE_USAGE	User → referenced type	Field/return types, allocation, annotations and related type evidence
EXCEPTION_THROWS	Declaring class → exception type	Declared exception metadata

All seven categories are enabled with weight one. Repeated evidence is deduplicated per ordered pair and category; the evaluated graph is their **binary directed union**. Merged self-relations are removed. A class with no remaining edges remains a node. Category overlap does not multiply edge weight. Member-use records are separately preserved for Usage Equality. [TUT p.6; D6]

Class processing	10.0	33.7.2
Owned binary classes before merging	1,157	1,944
Nested/anonymous binary identities merged	840	1,353
Final canonical internal classes	317	591
Non-owned production binary classes excluded	0	4

The internal boundary is enforced at both ends

Both endpoints must be in the owned node set. JDK, failureaccess and annotation dependency classes are excluded. The bundled `com.google.thirdparty.publicsuffix` namespace is outside the declared boundary even though it appears in Guava's own repository and JAR. Tests, testlib, GWT, Android and benchmark modules are not inputs. Stable-release APIs annotated `@Beta` remain internal; a beta API annotation is not a beta release. [D4; D6]

Multi-release JARs use base class entries only. Runtime reflection and configuration-created dependencies are not observed; virtual call expansion is not inferred. Missing or ambiguous declaring-member resolutions are recorded, not converted into invented member calls. These choices define a static view with known omissions. [L5 PDF pp.9,18–19; D6]

5 Replacement JNode: Usage Equality

JNode-UE implements Yano and Matsuo's Usage Equality method. It is the replacement requested for this assignment, grounded in the supplied 2020 paper. It is not a reproduction of the older JNode/Noise method based on ComponentRank and shortest-path significance. This methodological substitution is explicit. [UE pp.663–664; L67 pp.46–47; D2]

$$D_M(A,B) = \Sigma_{m \in M(A,B)} \, 1 / [xm_fanin(m) \times mx(B)]$$
$$e(B) = 1 - \max_A D_M(A,B)$$
$$c(B) = 1 - 1 / fanin(B)$$

Remove B exactly when **e(B) ≥ 0.85** and **c(B) ≥ 0.90**.

Term	Precise meaning after class-owner normalization
M(A,B)	Distinct members of B referenced by members belonging to caller class A.
xm_fanin(m)	Distinct source members outside B that reference target member m; not caller classes.
mx(B)	Number of B's distinct members referenced from outside B; unreferenced members do not enter the denominator.
fanin(B)	Number of distinct other internal project classes with resolved member use of B.

The cardinality threshold requires at least ten caller classes. A class without incoming member use has undefined scores and is retained. Scores are computed on the original graph once; all qualifying nodes are removed simultaneously. No repeated cascade or per-release threshold search is performed. “Outside B” means another owned class, not an external library. Every score, strongest caller and decision is saved. [UE; D2]

A small example explains the difference from degree

Suppose one utility operation is referenced by one member in each of ten caller classes. Each caller's dedication is 1/10, so e = c = 0.90 and the utility qualifies. Now imagine a logger shared by 21 callers, where a startup class alone also uses a separate initialization operation. Startup's dedication is 1/(21×2) + 1/(1×2), making e approximately 0.476. That exclusive relationship prevents removal despite high fan-in. This is a constructed explanatory example, not a Guava measurement. [UE Fig.1; D2]

Implementation: set/dictionary aggregation over distinct member references; cached declaring owners; stable summation and exact rational fallback at threshold boundaries. Expected core aggregation is O(R + C + N), plus deterministic output sorting, where R is unique member references, C caller–target class pairs and N classes. No all-pairs shortest paths are required. [D2]

6 Two recovery methods, frozen rules

ACDC: the original implementation

The archived York Java implementation is compiled and invoked using `+so -l20`. SubGraph Dominator runs before Orphan Adoption. The BodyHeader pattern matches C source/header pairs and provides no Java class-level evidence here. The default fine, flat DownInducer output is preserved. [TUT pp.7–8; L67 pp.19–24; ACDC; D7]

The maximum-size parameter governs SubGraph construction, **not every final cluster**. ACDC can place remaining orphans into a large final container. This output is reported without post-hoc splitting. Dependency-only RSF cannot encode isolates, so known genuine isolates are restored as explicit singleton clusters. All other classes must appear exactly once in ACDC's output. Invalid, duplicate, missing or stale output is rejected. [D7]

KMeans: fixed structural coordinates

$$U = \text{sorted union of class identities across all 30 releases}$$
$$X = \text{row-L2-normalize}([A + I, (A + I)^T])$$
$$k_{full} = \max(1, \text{ceil}(\sqrt{(n_{full}/2)}))$$
$$k_{filtered} = \min(k_{full}, n_{filtered})$$

Each current class is a row in a sparse CSR matrix with **1,424 fixed columns** (two directions × 712 union identities). Absent or removed coordinates contain zero evidence. Identity entries encode self-similarity only in features; they never enter the dependency graph or metric calculation. Package names and AI cluster labels are not features. [D7]

The square-root k rule is a transparent engineering heuristic, not a lecture theorem or a guaranteed optimum. It avoids a k sweep and avoids choosing k by the same MQ used for evaluation. A3/A4 share the full-release-derived k, subject to the retained-node limit. Empty graphs use k=0. Different versions may therefore have different k while obeying the same rule. [D7]

KMeans uses k-means++ initialization, Lloyd updates, random_state=484, n_init=10, max_iter=300 and tol=10^{-4}; numerical thread pools are limited to one. Class order and output labels are canonical. Actual k, convergence warnings, iterations, inertia and library versions are recorded, so identical rows producing fewer clusters cannot go unnoticed. [SK; D7]

7 Pilot and final parameter decision

The pilot uses **Guava 28.2**, the middle selected release by sample index, chosen before examining its metrics. ACDC is inspected with maximum subgraph sizes 10, 20 and 40. KMeans is inspected once using the same size-derived k rule planned for the full study. The pilot is recorded separately from the 120 final architectures. [TUT pp.7–8; D8]

Pilot configuration	k	Largest	Singles	Cohesion	Coupling	MQ
ACDC -l10	90	44	3	0.385	0.006	0.379
ACDC -l20	90	44	3	0.385	0.006	0.379
ACDC -l40	90	44	3	0.385	0.006	0.379
KMeans (fixed rule)	17	60	0	0.089	0.012	0.077

All three ACDC pilot settings produce the same cluster count and required quality metrics. This particular pilot provides no evidence that changing the subgraph limit improves the result; the documented middle setting is retained. Equal metrics alone should not be mistaken for a general parameter-insensitivity theorem.

What was frozen

The final study retains ACDC +so/-l20 and the predeclared KMeans rule. Coverage, emitted cluster sizes and output validity are checked explicitly. A larger MQ in one pilot trial would not establish a generally better architecture or justify tuning each release independently. The pilot demonstrates the consequences of the parameter, including ACDC's later orphan container, rather than treating -l as a hard global size cap. [D8; D7]

A representation problem found and repaired

The original file-oriented ACDC strips the final dotted suffix when deriving a basename. Bare Java names such as p.A and p.B would therefore share basename p and could collapse distinct output containers. The adapter now reversibly passes p.A.java and p.B.java, then decodes output membership back to the real class names. Every edge is mapped one-to-one, and node_id_map.csv records the translation. The original algorithm source is unchanged. The failed preliminary representation was rejected before the final study. [D8; D7]

There is **no per-release k search**, no threshold search and no collection of multiple candidate architectures from which the most favorable final MQ is reported. The same relation policy, normalization, UE thresholds, algorithms, seeds and evaluation formulas are used across the selected history. One configuration file and its hash bind those rules to the recorded run. [D8]

Frozen analysis configuration SHA-256:

`c9cdafab5f35e5df2d1b9947a5b3fd0f3b5fca5cc51b60f26de1c64e2a8c6376`

Pilot evidence: results/pilot/pilot.json; acdc_l10.csv; acdc_l20.csv; acdc_l40.csv; kmeans.csv. The JSON stores all per-cluster metric terms and actual execution metadata, including any output warnings. Final run metadata are under results/versions/<version>/.

8 Exact lecture quality measures

The report uses the modularization-quality definition shown in the reconstruction lecture: mean cluster cohesion minus mean coupling over unordered cluster pairs. Edges are directed, binary and without self-loops. Let n_i be cluster size, μ_i its internal edge count and ε_{ij} the inter-cluster edge count in both directions. [L67 pp.15–17]

$$\text{cohesion}_i = \mu_i / [n_i(n_i - 1)] \text{ for } n_i > 1; \text{ otherwise } 0$$
$$\text{coupling}_{ij} = \varepsilon_{ij} / (2n_i n_j)$$
$$MQ = (1/k)\ \Sigma_i\ \text{cohesion}_i - (1 / \text{choose}(k,2))\ \Sigma_{i<j}\ \text{coupling}_{ij}$$

The lecture presents alternative internal-density denominators; this study follows the n(n−1) form used in its worked directed-graph example. A singleton's cohesion is zero. For k=1, coupling is zero. Empty partitions report zero by explicit convention. Every zero-edge cluster pair remains in the coupling average. These policies are declared because otherwise software implementations can compute different values from the same partition. [L67 p.17; D9]

A hand-calculable oracle

Take clusters C1={a,b} and C2={c,d}, with edges a→b, b→a, c→d, d→c and b→c. Each cluster has cohesion 2/(2×1)=1. Pair coupling is 1/(2×2×2)=1/8. Therefore **MQ=1−1/8=0.875**. Adding a separate isolated singleton gives mean cohesion 2/3 and mean coupling (1/8)/3=1/24, hence MQ=5/8. The changed score comes from the declared averaging rule, although the original five edges are unchanged. [D9]

What the numbers do and do not establish

MQ is bounded between −1 and 1 for this model. It measures structural concentration, not functional correctness, runtime performance, developer productivity or user-perceived quality. Macro averaging gives a tiny cluster the same weight as a large one; changing k, isolate handling or cluster sizes can change the score. Node-size-weighted cohesion, edge fractions and cluster-size distributions are therefore reported as diagnostics. [L67 p.16; L12 p.45; D9]

A1 and A3 share the full graph; A2 and A4 share the retained graph. Direct full-versus-filtered MQ differences change both the grouping and the population. The same-graph comparison later projects the full partition onto exactly the retained nodes and edges before evaluating it. Adjusted Rand agreement compares partitions without treating either as ground truth. [D3; D9]

9 Classes and dependencies over time

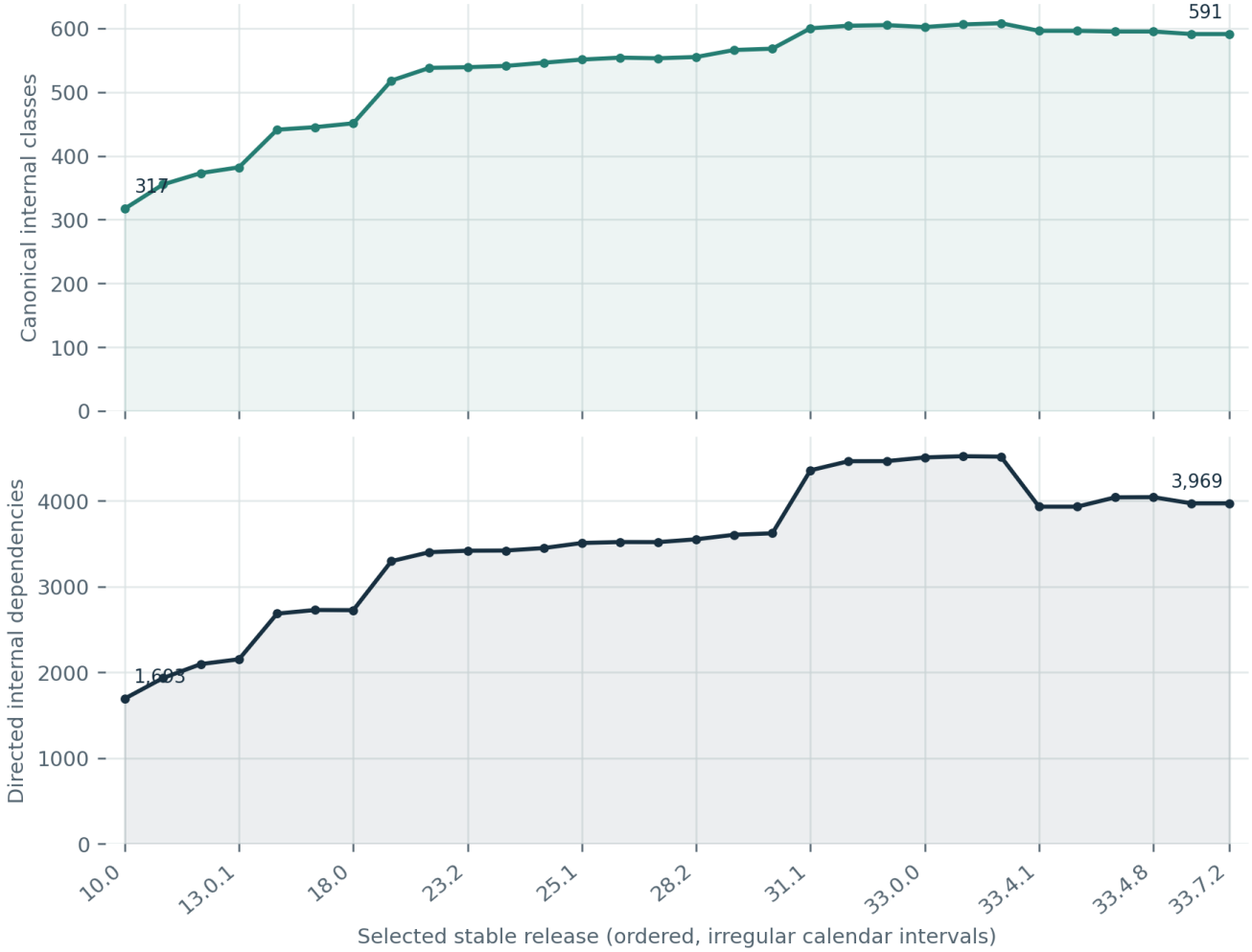


Figure 1. Canonical internal classes and directed binary dependencies. Both panels use the same release order and separate, labeled vertical scales. Source: results/sizes.csv.

The first selected release contains 317 classes and 1,693 dependencies; the last contains 591 classes and 3,969 dependencies. These counts are taken after owner normalization and removal of external endpoints, so they are directly tied to the graph used for A1/A3. Raw class-file totals would overcount nested and anonymous implementation details. [D1]

Annotation evidence explains much of the visible rise and fall. From 30.1.1 to 31.1, 748 of 792 added edges target annotation types. From 33.3.1 to 33.4.1, 566 of 587 removed edges target 15 removed internal annotation types. The pinned source example moves nullness marking to external JSpecify annotations and package metadata, both outside this class graph. This is evidence of a representation/boundary effect, not a measured reduction in functional coupling. [D15]

The horizontal axis is a sequence of sampled releases. Equal visual spacing does not mean equal calendar duration: older major releases and newer patch releases have different time gaps. Counts alone cannot establish how much user-visible functionality was added. [L12 pp.43–44; D4]

10 Complete thirty-release inventory

Release	Published	Classes	Edges	UE out	Edges left	Skipped
10.0	2011-09-28	317	1,693	6	1,433	—
11.0	2011-12-18	355	1,932	9	1,580	1
12.0	2012-04-30	373	2,096	8	1,746	2
13.0.1	2012-08-28	382	2,152	8	1,795	2
15.0	2013-09-06	441	2,684	11	2,099	2
16.0.1	2014-02-03	445	2,726	10	2,138	1
18.0	2014-08-25	451	2,724	10	2,136	1
21.0	2017-01-12	518	3,295	16	2,454	2
23.0	2017-08-04	538	3,400	17	2,515	1
23.2	2017-10-11	539	3,417	17	2,524	1
23.5	2017-11-22	541	3,419	16	2,553	2
24.0	2018-02-01	546	3,448	17	2,563	1
25.1	2018-05-31	551	3,507	17	2,611	4
27.0	2018-10-18	554	3,518	18	2,577	1
27.1	2019-03-08	553	3,517	18	2,578	1
28.2	2019-12-27	555	3,550	19	2,601	2
30.0	2020-10-16	566	3,603	19	2,703	1
30.1.1	2021-03-19	568	3,620	20	2,681	1
31.1	2022-02-28	600	4,354	22	3,302	2
32.0.1	2023-06-09	604	4,460	22	3,402	1
32.1.2	2023-07-31	605	4,461	22	3,403	2
33.0.0	2023-12-18	602	4,504	23	3,429	1
33.2.0	2024-05-02	606	4,518	23	3,440	1
33.3.1	2024-09-23	608	4,513	24	3,304	2
33.4.1	2025-03-19	596	3,930	23	2,810	1
33.4.3	2025-03-19	596	3,930	23	2,810	1
33.4.6	2025-03-25	595	4,039	23	2,917	2
33.4.8	2025-04-14	595	4,041	23	2,919	1
33.7.0	2026-08-17	591	3,969	22	2,886	2
33.7.2	2026-09-29	591	3,969	22	2,886	1

Classes are canonical owned nodes; edges are distinct directed dependencies. “UE out” counts removed nodes. “Skipped” counts numeric stable tags strictly between the current and previous sampled release, including any explicitly excluded mainline/backport point in that interval. Exact intervening names, Git SHAs and artifact variants are in config/versions.json. [D1; D4]

11 What Usage Equality removes

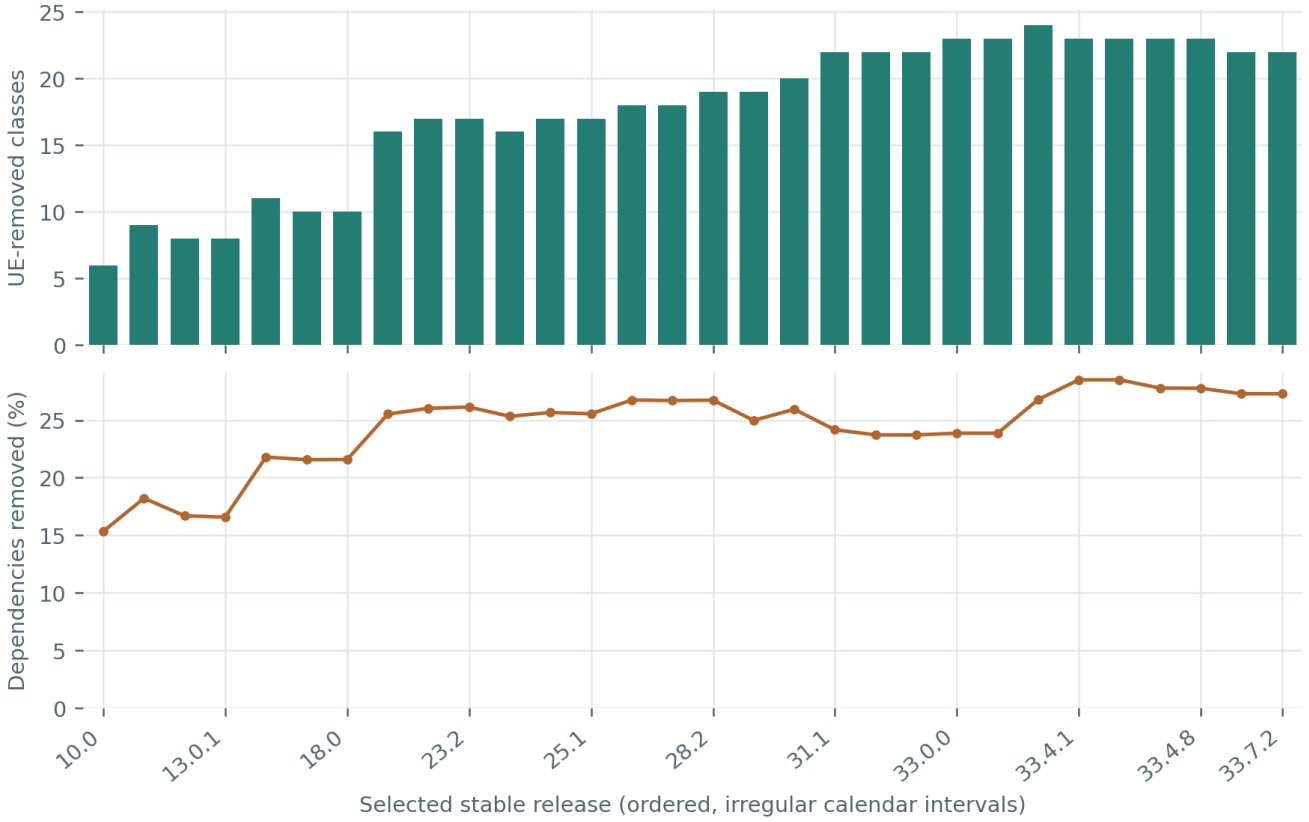


Figure 2. Detected omnipresent nodes and the fraction of original graph edges incident to removed nodes. Removal is simultaneous and uses fixed thresholds. Source: results/sizes.csv.

Frequently removed class identity	Releases detected / 30
base.Function	30
base.Preconditions	30
collect.Serialization	30
collect.UnmodifiableIterator	30

At the latest release, UE removes 22 of 591 classes and leaves 2,886 dependencies (72.7% of the full graph). A frequent removal is not proof that a class is badly designed. A utility may be useful and correct while obscuring subsystem structure for a clustering algorithm. [UE; D1; D2]

Frequency counts use all 30 releases as the denominator, including releases in which an identity does not yet exist. Full per-class scores distinguish absent classes, retained candidates and detected classes. See results/versions/<version>/ue.json and omnipresent.csv.

Latest audited target	Callers	Members	e	c	Decision
base.Preconditions	312	29	0.9276	0.9968	Remove
collect.Iterators	50	41	0.4980	0.9800	Retain

Preconditions passes both thresholds. Iterators has many callers but a strong specialized relationship with Iterables, giving a maximum dedication of about 0.502 and preventing removal. The independent case audit preserves exact fractions, all caller scores and member-CSV row evidence; names alone do not determine these decisions. [D14]

12 Modularization quality results

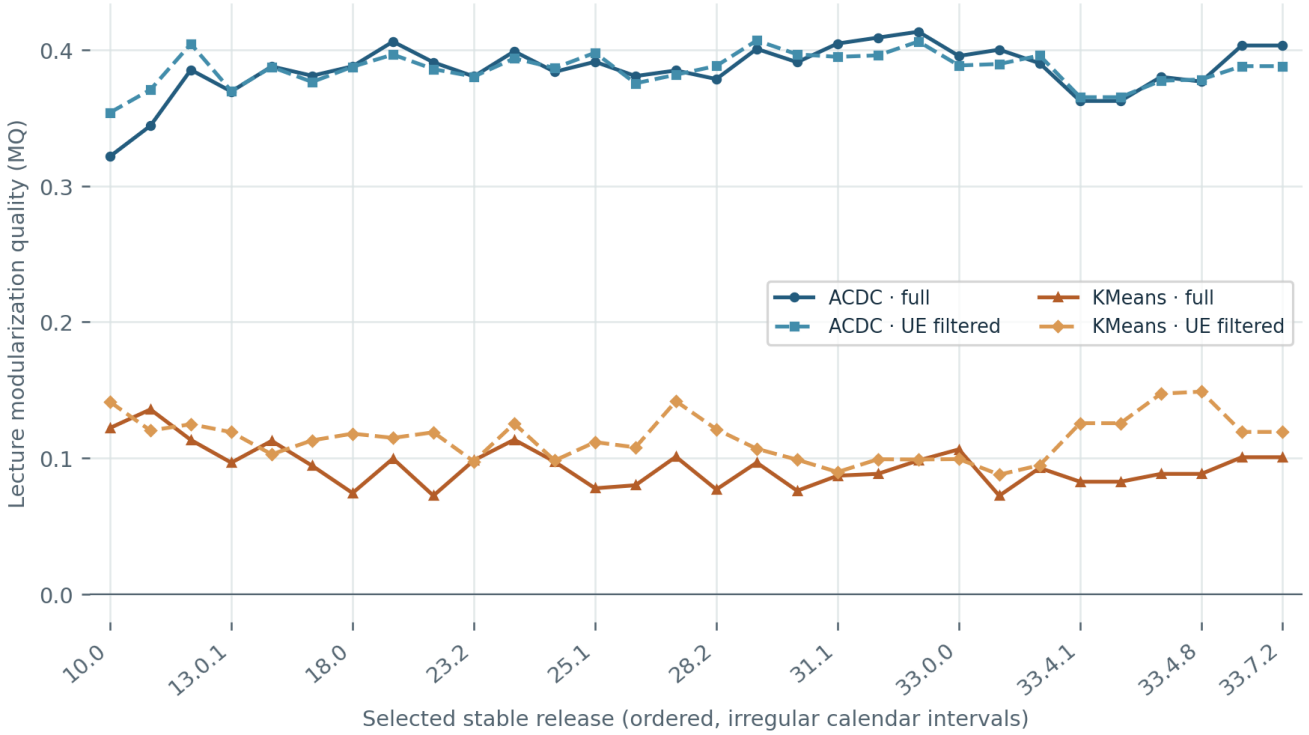


Figure 3. The four required MQ series. A1/A3 evaluate full graphs; A2/A4 evaluate retained graphs. Cross-graph gaps are descriptive and must not be read as a causal filtering benefit.

Condition	Minimum	Median	Maximum	Latest
ACDC · full	0.322	0.388	0.413	0.403
ACDC · UE filtered	0.354	0.388	0.407	0.388
KMeans · full	0.072	0.095	0.136	0.101
KMeans · UE filtered	0.088	0.116	0.149	0.119

On the same full graph, ACDC has higher MQ than KMeans in 30 of 30 releases; on the same filtered graph it has higher MQ in 30. This comparison describes the chosen metric and granularity rules. It does not demonstrate that the higher-scoring partition matches the intended subsystem architecture. Cluster counts and sizes are examined alongside the score. [D1; D9]

13 Cohesion and coupling separately

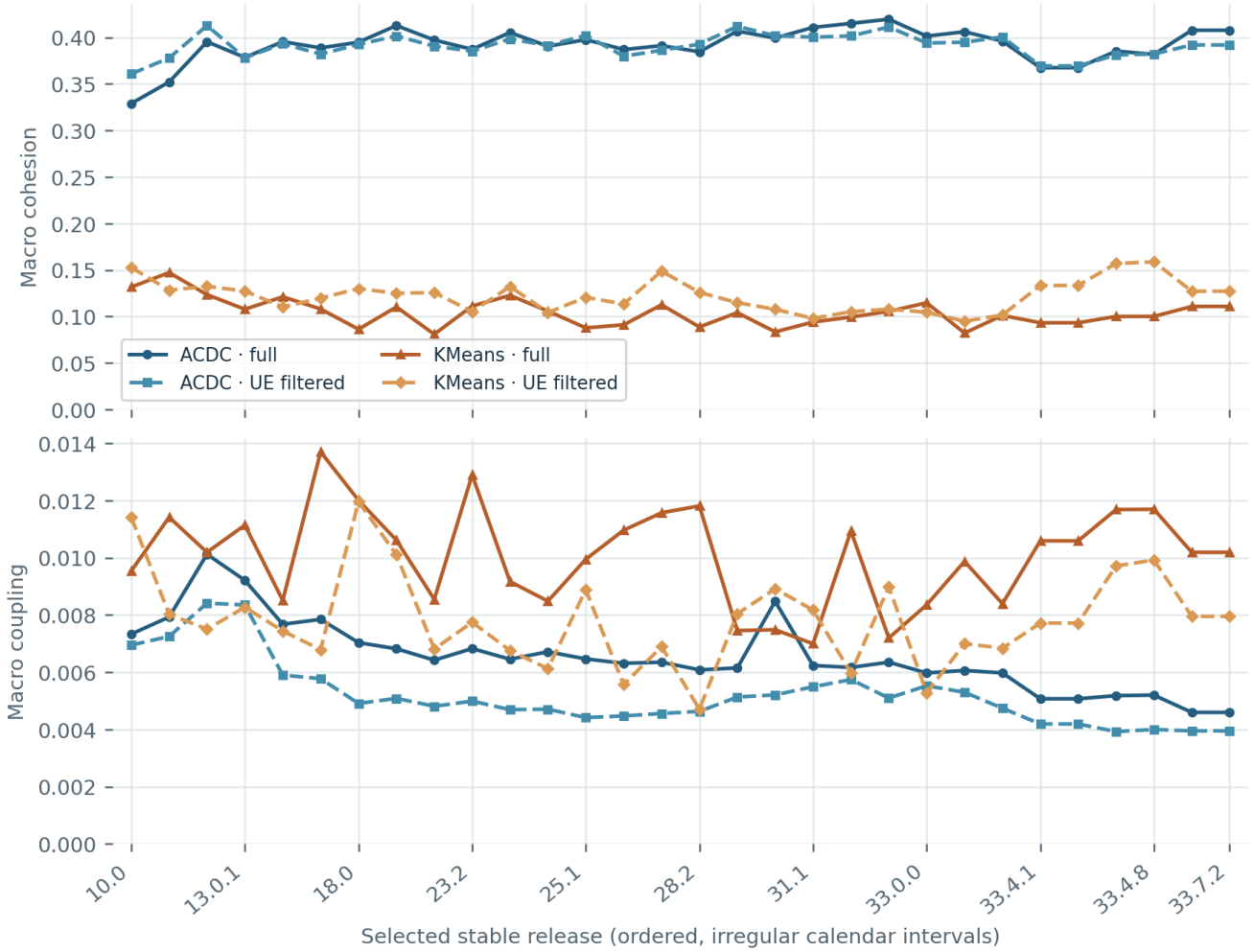


Figure 4. Required macro cohesion and macro coupling. Each panel has its own vertical scale. Both quantities average over clusters or cluster pairs, not over classes.

Showing the terms separately prevents a single MQ value from hiding its composition. A change can come from denser internal groups, fewer cross-group edges, or a change in how cluster and pair sizes are weighted. The coupling denominator also grows with the number of cluster pairs. A low macro coupling score is therefore not equivalent to having few inter-cluster edges. [D9]

For an oral explanation, choose any appendix row and open its corresponding A<n>_quality/per_cluster.csv and per_pair.csv. Sum the saved normalized terms, divide by the numbers of clusters and unordered pairs, and subtract. The report's rounded values are display values; the CSV and JSON outputs preserve the computation precision.

14 Does filtering change the grouping?

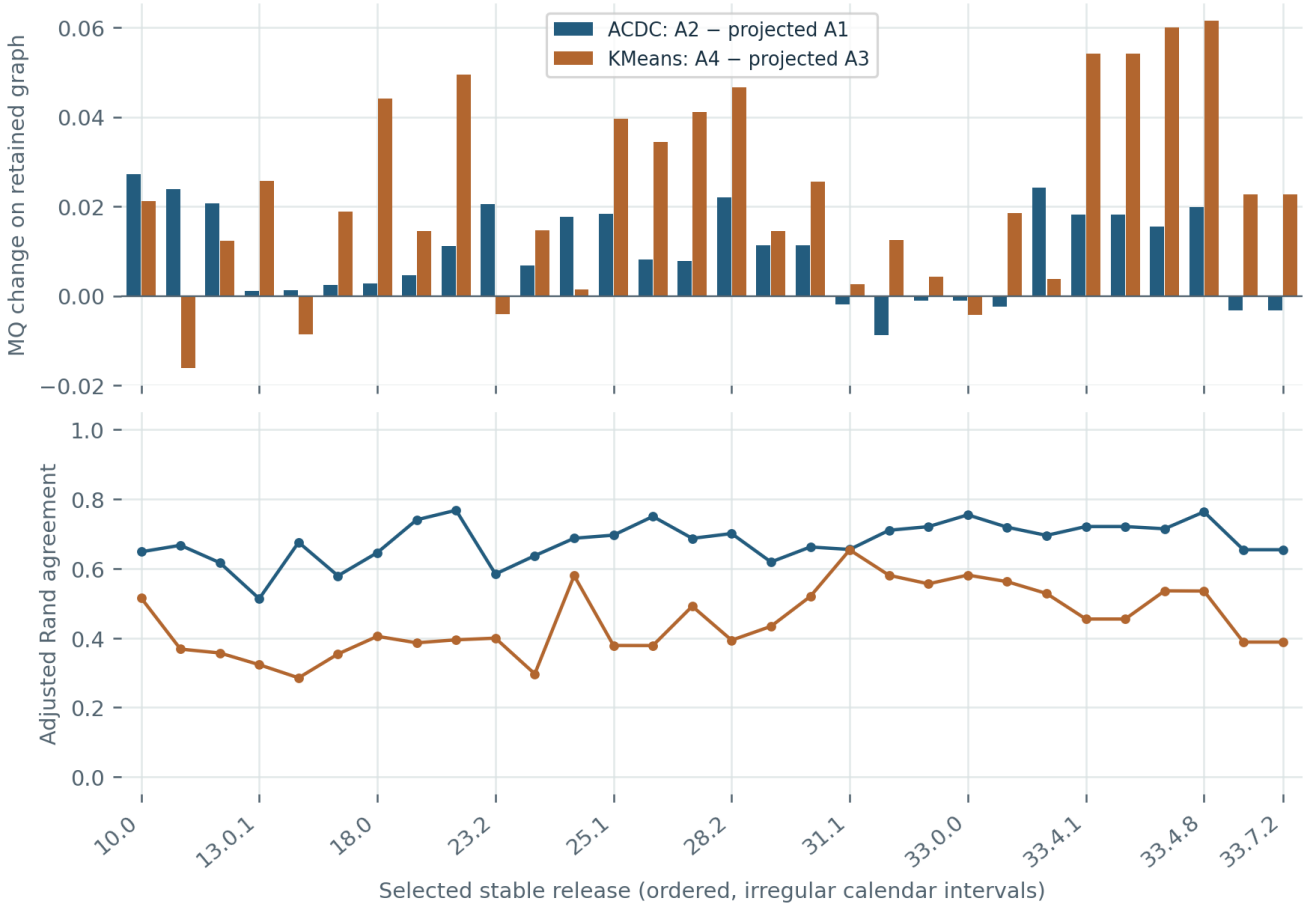


Figure 5. Recovered filtered partition versus the full partition projected to the identical retained graph. Positive MQ difference favors reclustering under the lecture metric. ARI is agreement, not correctness. Source: results/same_graph_comparisons.csv.

Algorithm	Median ΔMQ	Latest ΔMQ	Latest ARI	Δ > 0	Δ < 0
ACDC	0.0096	-0.0032	0.654	23	7
KMeans	0.0200	0.0226	0.388	26	4

Projection removes the same classes from the full partition without reclustering the survivors. Empty projected clusters disappear. Both projected and newly recovered assignments are then evaluated using the exact same retained edges. This controls the evaluated population, making the difference attributable to the grouping produced under the declared recovery process. It still does not supply an expert architecture or prove a general benefit of removal. [D3]

15 Granularity and metric sensitivity

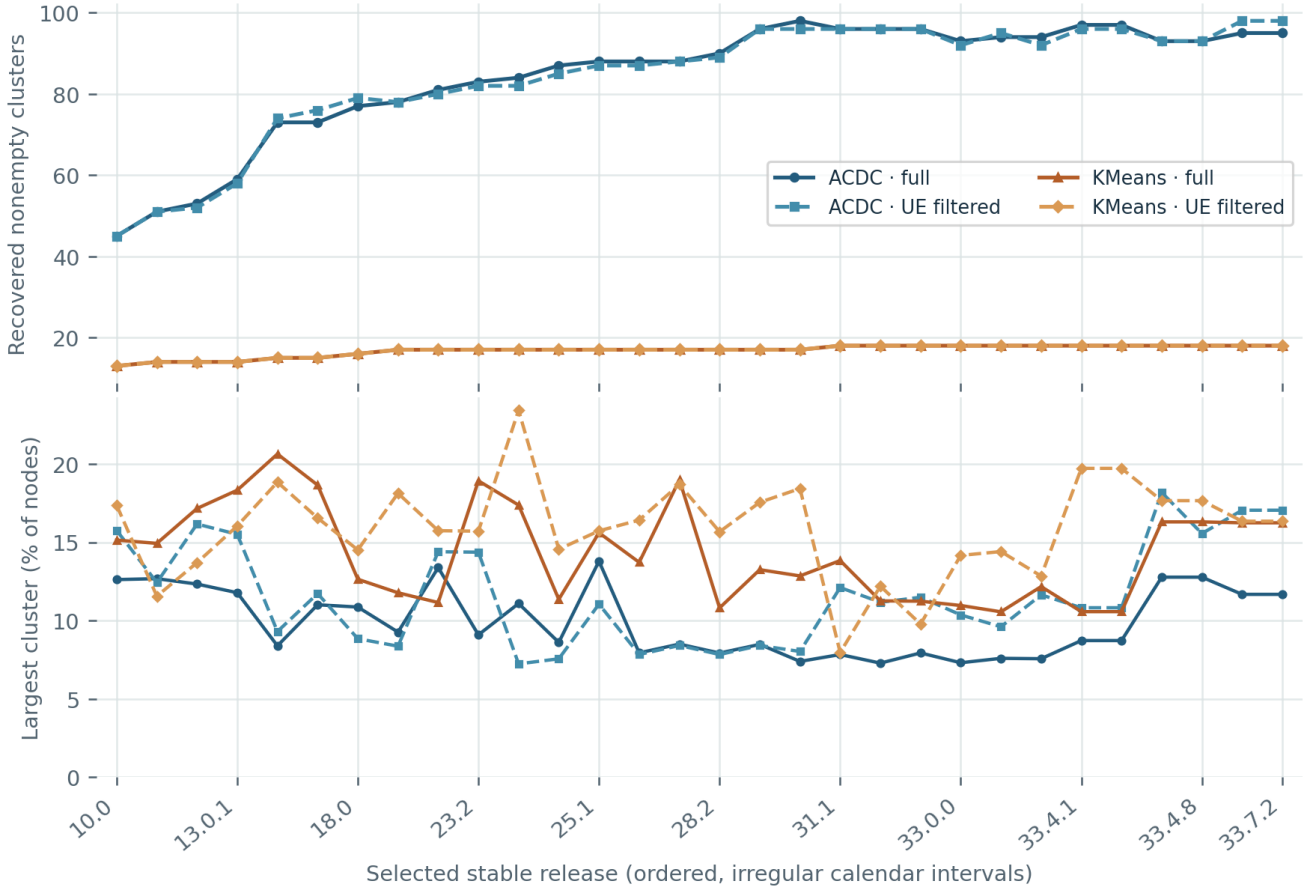


Figure 6. Number of recovered clusters and share of nodes in the largest cluster. ACDC's subgraph-size setting is not a final cluster-size cap.

Latest	k	Largest	Singletons	Size-weighted cohesion	Inter-edge share
A1	95	69	4	0.244	69.9%
A2	98	97	5	0.241	64.6%
A3	18	96	0	0.068	74.9%
A4	18	93	0	0.075	66.5%

The supplementary cohesion weights each cluster by its node count, while the required MQ weights each nonempty cluster equally. Inter-edge share counts all edges crossing the partition boundary and divides by the total edge count. Neither diagnostic is substituted for the lecture metric. Their purpose is to reveal large residual containers or numerous tiny clusters that the macro averages can conceal. [D9]

16 Structural change between samples

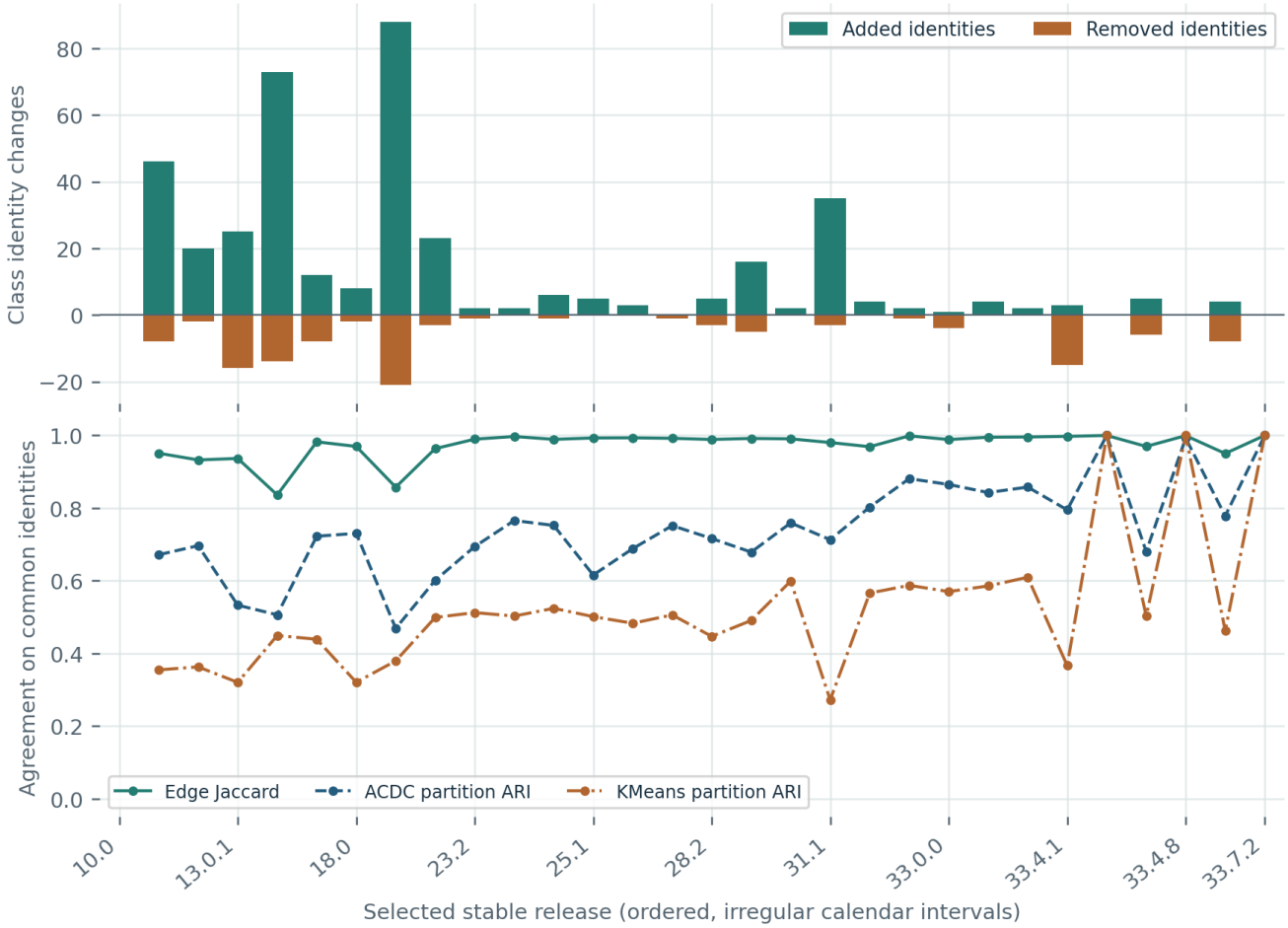


Figure 7. Added/removed canonical identities and structural agreement between adjacent selected samples. Agreement uses only class identities common to the pair.

The largest sampled identity turnover is 18.0 → 21.0: 88 added and 21 removed identities. An identity change may represent introduction, deletion, renaming, package migration or a boundary effect; it is not automatically a new or removed feature. [D10]

A concrete boundary example is TrieParser: its com.google.common.net identity is internal in 15.0, while the later com.google.thirdparty.publicsuffix namespace falls outside this study. That difference must not be narrated as disappearance of its functionality. Sparse release sampling also cannot isolate the particular commit responsible for a transition. [D4; D6]

At 33.3.1 → 33.4.1, common-node edge Jaccard is 0.9975 despite the large total edge decrease. Edges touching removed annotation nodes do not enter that common-node comparison. The identical graphs at 33.4.1/33.4.3 and 33.7.0/33.7.2 also produce identical final partitions under the fixed rules. Total size and agreement on surviving identities therefore answer different questions. [D10; D15]

17 Reading the latest recovered groups

A partition is more explainable when its members are inspected. The tables below show the five largest clusters from each full-graph recovery at the latest release. Package composition is computed after clustering as a descriptive aid; package labels were not algorithm features. Cluster identifiers are canonical output labels, not authoritative subsystem names. [D7]

ACDC · full

Cluster	Nodes	Dominant packages; example identities
acdc_0001	69	collect: 34; util.concurrent: 7; base: 5 annotations.GwtCompatible, annotations.GwtIncompatible, annotations.J2ktIncompatible
acdc_0016	40	collect: 36; base: 2; math: 1 base.Predicate, base.Predicates, collect.AbstractListMultimap
acdc_0002	39	hash: 26; primitives: 6; base: 3 annotations.VisibleForTesting, base.Preconditions, base.Strings
acdc_0011	28	util.concurrent: 27; base: 1 base.Function, util.concurrent.AbstractCatchingFuture, util.concurrent.AbstractFuture
acdc_0048	26	graph: 23; collect: 3 collect.HashBiMap, collect.HashMultiset, collect.UnmodifiableIterator

KMeans · full

Cluster	Nodes	Dominant packages; example identities
kmeans_0002	96	collect: 31; util.concurrent: 22; base: 20 base.Absent, base.CaseFormat, base.Charsets
kmeans_0005	62	io: 23; util.concurrent: 18; math: 8 base.Defaults, base.Enums, base.JdkPattern
kmeans_0000	61	collect: 12; util.concurrent: 11; base: 9 annotations.Beta, base.CommonMatcher, base.CommonPattern
kmeans_0008	52	collect: 52 collect.AbstractBiMap, collect.AbstractIndexedListIterator, collect.AbstractListMultimap
kmeans_0011	51	collect: 48; net: 1; reflect: 1 collect.ArrayTable, collect.CollectCollectors, collect.ConcurrentHashMultiset

For ACDC, the saved original_containers map links normalized labels to the implementation's container names, including any orphan container. Inspect this map before assigning a semantic meaning to a large group. Complete assignments remain available for every condition and release. [D7]

18 Latest-release AI experiment

Recorded model: ChatGPT GPT-6 Astra Pro (fresh delegated session; no seed control exposed). The subject is the same pinned Guava 33.7.2 release used by the classical experiments. A fresh delegated session first wrote a qualitative hypothesis from class names and package counts, then read the other evidence and produced one complete final partition. The initial hypothesis is not a second finalized architecture. [AS pp.3–4; D12]

Exact supplied input	Bytes	SHA-256 prefix
classes.csv	80,378	9a8bad80015b9b51
dependencies.csv	347,898	400a2343e5d0bef1
package_counts.json	586	ae7261c475015834
package_dependencies.csv	4,275	319d039818af035f
package_documentation.txt	23,935	66ba51ddb5659017

The provided graph contains 591 classes and 3,969 directed dependencies. Package documentation is taken from the corresponding source JAR. The protocol withholds ACDC/KMeans partitions, UE decisions and metric outcomes, forbids external browsing, and requires complete one-module-per-class coverage before the coordinator calculates quality. Exact full hashes, paths, activity and outputs are preserved. [D12]

Information provided and reproducibility limits

Source names and package documentation are allowed AI evidence, whereas structural algorithms receive graph features only. This information asymmetry is intentional and prevents treating this as a controlled same-input superiority test.

Fresh session still has general pretrained knowledge; hidden system instructions and model nondeterminism cannot be fully reproduced.

The initial hypothesis and the final explanation are saved separately as initial_hypothesis.md and response.md. The following pages reproduce the actual prompt, enumerate the returned modules, and evaluate that fixed output. Replaying the coordinator's calculations is deterministic; rerunning a live model need not reproduce the same proposal. [D12]

The original AI JSON is preserved: its top-level version field is “1.0”, while subject.release identifies Guava 33.7.2. The experiment record and pinned inputs define the analysed software release. [D12]

Additional user-visible messages, verbatim

The primary task prompt is reproduced on the next page. These are the exact launch instruction and the later completion/status follow-up; the latter refers to the model's already established proposal. Neither supplies a baseline quality result. [D12]

Launch instruction: Working directory /workspace/scratch/7500a23b5952/software-evolution-ex1. This is a controlled, actual AI architecture-recovery experiment. Your exact user task is the full contents of results/ai/prompt.txt. Read that file first and follow it strictly. Read only that prompt and the listed results/ai/inputs evidence; do not inspect any other project data/results/code, baseline architectures, UE outputs or numeric quality scores. Produce the specified outputs under results/ai. Your launch identity is GPT-6 Astra Pro as inherited. The launcher supplied no algorithm outcomes. Complete this now; another reviewer will evaluate afterward.

SHA-256: 390b4c25c11f2804ce9698685d3c70ecc97d884729bfa9c0eca4fd9a7d36c9ad

Coordinator follow-up: Status check: all core study and dataset are committed; only your final experiment and report integration remain. Please finish and save the requested outputs now using your established evidence-based18-module proposal, with exact coverage. No further refinement/search is needed. Root evaluator is ready and continues to withhold all quality scores from you.

SHA-256: 04e3586c1dfffbe2595db528103c084117cb64fbc57587b6c1ddfc495c2f5cb9

19 Exact prompt supplied to the AI

The text below is the saved experiment prompt, reproduced without editorial changes. Hidden platform instructions and model internals are not claimed to be part of this record. [D12]

Recover a maintainable conceptual module architecture for the latest Guava JRE release, 33.7.2, from the supplied evidence. This is an actual recorded AI architecture-recovery experiment for EPL484 Software Evolution. You are a fresh AI reviewer and must not inspect any ACDC/KMeans partitions, UE decisions, metric outcomes, reports, or other experiment outputs.

Use only the following files under results/ai/inputs (all from the same pinned release): classes.csv (591 canonical outer classes), dependencies.csv (3,969 binary directed internal class dependencies), package_counts.json, package_dependencies.csv, and package_documentation.txt (unmodified package documentation from the corresponding source JAR). External libraries have already been excluded and inner/anonymous classes merged. A dependency points from the class using a type/member to the class being used. Package declarations and documentation are evidence, not a validated architecture ground truth. Do not browse external sources or inspect other project files. You may use Python to read and summarize only these inputs and to produce the requested outputs.

First inspect classes.csv and package_counts.json only. Record a concise initial architecture hypothesis in results/ai/initial_hypothesis.md before reading dependency/documentation inputs. This initial hypothesis is qualitative, not a second finalized architecture. Then read the remaining evidence and refine the hypothesis into ONE complete final class partition. Select a practical number of named responsibility modules; do not target a numeric clustering metric. Classes in one package may be split if a specific responsibility boundary and dependency evidence justify it. Preserve low-degree/isolated types. Give every supplied class exactly one module and invent no classes. Do not place a class in multiple modules, including utility classes.

Write these outputs under results/ai only:
1. assignment.csv with columns class,cluster, containing every supplied class exactly once. Semantic stable cluster IDs are preferable.
2. architecture.json with version, modules (id, name, responsibility, exact class_count, boundary_rationale), cross_module_observations, changes_from_initial_hypothesis, and limitations. Cite concrete class/package relationships and counts from the supplied graph where relevant, clearly separating interpretation from observation.
3. response.md: your complete final explanation, including how the additional evidence changed or confirmed the initial names-only hypothesis; whether package structure biased your reasoning; confidence limits; and which intended-design claims would require maintainer validation. Do not claim superiority or architectural accuracy without ground truth. Explain why this proposal helps a human understand Guava.
4. activity.json: exact input paths read, tool/method summary, model identity available from your runtime (do not invent a precise model version if unavailable), and confirmation that baseline partitions and numeric quality scores were not consulted.

Validate partition coverage against classes.csv. Do not compute MQ/cohesion/coupling or compare with the baseline algorithms; the experiment coordinator will evaluate the final partition afterward on matching graph scopes. Leave all unrelated files untouched.

`Prompt SHA-256: 73bad8434d1d4e43075eb783f81cf10442342d8fc2b0389fd08ad7fd219c6464`

20 The AI's proposed conceptual modules

The finalized AI assignment contains **18 modules covering all 591 supplied classes exactly once**. Responsibilities below are the model's interpretation of the supplied names, documentation and dependencies. They are not maintainer-approved design constraints. The machine-readable architecture also records each boundary rationale and its supporting observations. [D12]

Module	Classes	Responsibility excerpt
Foundation and common contracts	57	Basic contracts, validation, optional values, functions and predicates, equivalence, general string utilities, timing, reference cleanup, and shared…
Primitive values and arrays	25	Per-primitive utilities, unsigned numeric value representations, primitive type adaptation, and immutable primitive arrays.
Arithmetic and statistics	16	Arithmetic, rounding, numerical preconditions, summary statistics, quantiles, and linear statistical relationships.
Core collections and shared collection machinery	125	Lists, sets, ordinary and bidirectional maps, raw class-to-instance maps, collection views and decorators, immutable implementations of those types,…
Multimaps and counted collections	59	Mappings with multiple values per key, collections that count repeated elements, and their mutable, immutable, filtered, sorted, forwarding, and builder…
Tabular collections	16	Values addressed by row and column keys, including table views, mutable and immutable implementations, and table-specific collectors.
Intervals and bounded collections	15	Interval endpoints and bounds, discrete domains, contiguous sets, range sets, and range maps.
Caching and cache policy	22	Cache construction, loading, eviction/removal policy, statistics, and local cache implementation support.
Futures and asynchronous execution	57	Future results and composition, callbacks, listening executors, task execution and sequencing, thread creation, asynchronous failure types, and the shared…
Service lifecycle management	8	Start/stop service lifecycles, execution-thread and scheduled service templates, service grouping, and lifecycle listener delivery.
Synchronization and execution limits	18	Atomic state, locks and monitors, blocking collection adapters, interruption-tolerant waits, rate limiting, and time limits.
Graph models and traversal	53	Graph, value-graph, and network APIs, builders, mutable/immutable representations, connection storage, and traversal.
Hashing and probabilistic structures	33	Hash functions, hash values, streaming hashers and funnels, hash-aware stream wrappers, and Bloom filters.
I/O, resources, and binary encodings	37	Byte and character sources/sinks, stream and file/resource operations, resource closing, and binary/base encodings.
Escaping and format-specific escapers	16	Character/Unicode escaping mechanisms and HTML, XML, and URL specializations.
Network and protocol values	7	Host/port, IP address, domain-name and media-type handling, and HTTP header names.
Reflection and generic type modeling	16	Generic type capture/resolution, reflective invocation, type-indexed instances, classpath inspection, and reflection helpers.
Event registration and dispatch	11	Publish/subscribe event registration, dispatch, subscriber handling, and subscriber failure context.

Complete class-level output: results/ai/assignment.csv. Complete semantic response: results/ai/response.md. Initial hypothesis: results/ai/initial_hypothesis.md. Machine-readable module definitions, refinements and limitations: results/ai/architecture.json. [D12]

21 AI quality and comparison

Partition	Graph	k	Cohesion	Coupling	MQ	Inter-edge %
AI final	Full	18	0.1026	0.0040	0.0987	51.7%
A1	Full	95	0.4078	0.0046	0.4032	69.9%
A3	Full	18	0.1108	0.0102	0.1006	74.9%
AI projected	Retained	18	0.0970	0.0029	0.0941	46.5%
A2	Retained	98	0.3920	0.0040	0.3880	64.6%
A4	Retained	18	0.1272	0.0080	0.1192	66.5%

The AI's full assignment is evaluated on the same graph as A1/A3. For A2/A4 comparisons, the same AI assignment is projected onto retained nodes and evaluated on retained edges, without asking the model for another architecture. Cross-scope scores are not compared as though they measured the same graph. Information supplied to the methods also differs: the AI sees names and documentation, while KMeans receives structural features. [D12]

Compared baseline	Matching graph	Adjusted Rand agreement
A1	full	0.2020
A2	retained	0.1870
A3	full	0.1890
A4	retained	0.1995

Three concrete refinements from the added evidence

Four responsibilities: core collections (125), multimaps/counted collections (59), tables (16), and intervals (15). Documentation distinguishes the data models; class edges connect interfaces to their variants and show 19 multimap-to-multiset references. Shared collectors and synchronization utilities explain why the proposal remains a conceptual division.

Futures/asynchronous execution (57), service lifecycle (8), and synchronization/execution limits (18). Service templates share the Service/AbstractService family; ListenerCallQueue has exactly the two lifecycle callers. Futures and executor adapters refer to each other, while lock/rate/time APIs express different controls.

Base text helpers remain in foundation; escape, html, xml, PercentEscaper, and UrlEscapers form a 16-class escaping module. The base graph interweaves Platform, validation, strings, and generic helpers. Documentation and graph edges explicitly connect all format-specific escapers to the common escape API.

Interpretation and confidence

AI MQ is 0.0987 on the full graph and 0.0941 after projection. AI and A3 both use 18 clusters, yet their adjusted Rand agreement is only 0.1890. Similar aggregate MQ therefore does not imply the same grouping. ACDC's higher score also accompanies many more clusters. These results evaluate structural concentration, while the model proposes a semantic reading map with reciprocal cross-module dependencies. No acyclic build-module decomposition or intended-design accuracy was validated. [D9; D12]

Partition agreement and coordinator evaluation details: results/ai/comparisons.csv; metrics.json; retained_metrics.json. The live model's complete limitations and cross-module observations remain in architecture.json and response.md. [D12]

22 Performance and scaling evidence

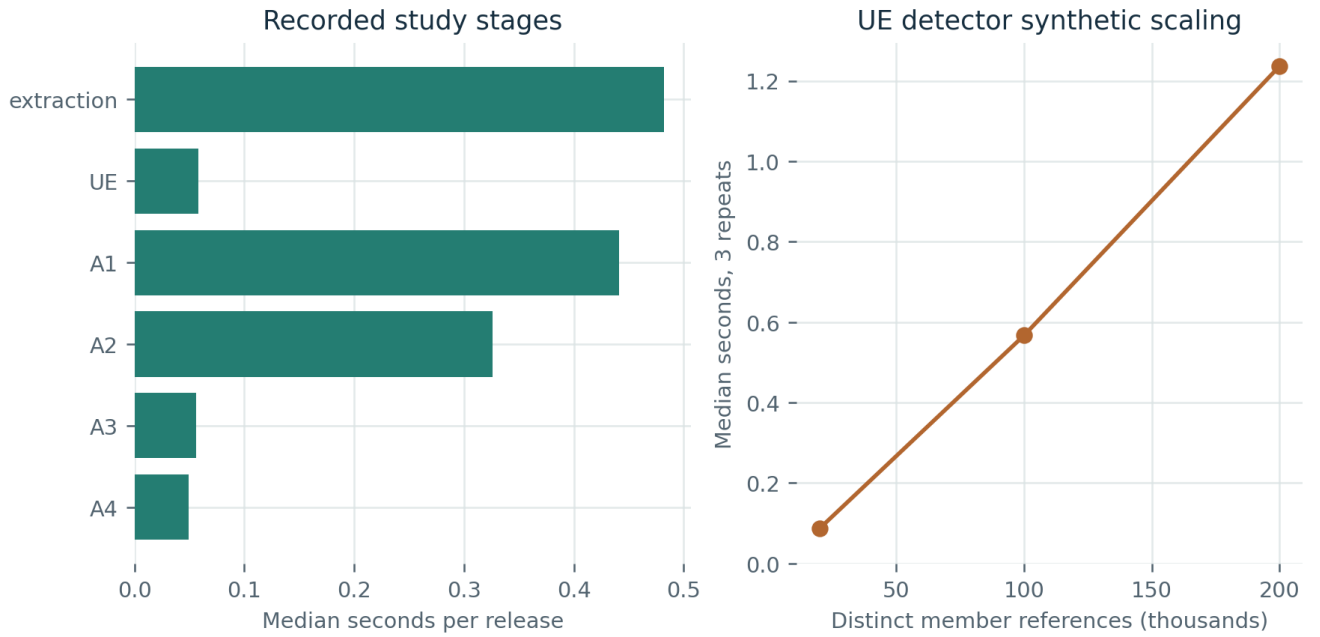


Figure 8. Actual study-stage timings and a separate controlled synthetic UE benchmark. They measure different scopes and are not speedup ratios.

Study stage	Median / release (s)	Maximum (s)	30-release sum (s)
extraction	0.4823	0.5769	14.138
UE	0.0579	0.1085	1.791
A1	0.4410	0.6092	13.504
A2	0.3263	0.4525	9.783
A3	0.0563	0.0751	1.685
A4	0.0495	0.0586	1.469

Synthetic N	Member references	Median (s)	Process high-water MiB
1,000	20,000	0.0875	34.0
5,000	100,000	0.5671	117.7
10,000	200,000	1.2360	221.9

Study extraction includes JVM startup and I/O; UE study time includes member-CSV parsing; clustering is wall time around each adapter. Downloads, the pilot, report generation and tests are not included in the stage sums. Synthetic timings isolate the detector including validation and diagnostics, use three repeats and fixed degree-20 indexed graphs, and verify expected scores for every generated node. [D11]

Memory is cumulative process high-water RSS, including constructed inputs and prior result objects; it is not per-call peak allocation. Times depend on this environment and do not prove a speedup over the old executable, which was not used as a benchmark or correctness oracle. The measurements support practicality within the tested input sizes. [D11]

23 Course-grounded interpretation

Implemented structure, intended design and drift

This is architecture reconstruction by reverse engineering: a lower-level static dependency graph is converted into a higher-level decomposition. The architecture-evolution lecture distinguishes implemented, intended and described architecture, as well as static and dynamic views. Our partitions are candidate views of the implementation. Demonstrating erosion would require an intended design constraint and evidence that changes violate it. MQ trends alone do not provide that evidence. [L5 PDF pp.9,12,14,18–19]

Lehman's laws are a lens, not an automatic conclusion

Changes in class counts and dependencies are consistent with continuing change and may motivate inspection of structural complexity. They cannot by themselves verify continuing functional growth, declining user-perceived quality, invariant effort or organizational stability. Those claims need requirements, usage, effort and process evidence. The lectures also restrict the empirical laws to E-type systems; a Java utility library should not be classified as E-type without explaining its real-world adaptation context. [L12 pp.33–48]

Operational definitions and traceability

The requirements lectures emphasize ambiguity, measurable statements and traceability. “Internal,” “class,” “dependency,” “stable release,” “omnipresent” and “quality” therefore have explicit operational definitions here. The same term can mean something different in requirements relationships, runtime interactions and class-file structure. Stored inputs, parameters and per-cluster terms make a claim auditable, but traceability alone does not guarantee its correctness. [L3 pp.9–19,27–33; L4 pp.11–15]

Material limits of this experiment

The 30 observations form a systematic sparse sample, not a random sample or a complete history. The artifact and namespace boundary omits external interactions and other modules. Static analysis cannot observe every reflective or dynamically created dependency. Published bytecode can reflect different compiler versions and inlining. The UE replacement depends on resolved member usage and its fixed thresholds; it is not interchangeable with the older Noise method. ACDC granularity and KMeans' geometry/heuristic affect the recovered structure. [D2; D4; D6; D7]

The latest extraction records 657 unresolved member-use events (492 distinct recorded references); 333 events cross normalized class boundaries. These diagnostics quantify a known evidence gap. Their ratio to raw observed calls includes external and intraclass events, so it is not a completeness or false-negative rate. [D14]

Automated metrics remove repeated manual calculation, but they do not remove modeling judgment. Without an expert architecture, partition agreement is not accuracy. The AI output adds a semantic interpretation and its own prompt/model constraints. The defensible conclusions are the observed structural results under these declared choices. [AS pp.3–4; L3 pp.16–19]

24 Reproduce and audit the study

Use the repository's pinned Python dependencies and Java toolchain instructions first. The following sequence reconstructs the data-to-report chain; --offline verifies an existing artifact cache. Exact flags and prerequisites are documented in the repository README and acquisition guide.

```
python scripts/unpack_data.py
python scripts/fetch_data.py --offline
python scripts/fetch_tools.py
python scripts/build_java.py
python scripts/run_study.py --phase all
python scripts/build_report.py
```

Artifact path	What to verify
config/versions.json	30 immutable release identities, URLs, hashes and actual skipped tags.
data/graphs/<version>/	Original/merged class aliases, seven typed relation categories, binary edges, member references, exclusions and unresolved evidence.
results/versions/<version>/	UE scores, retained graph, exactly one A1–A4 assignment and its execution/quality details.
results/*.csv	30 size rows, 120 metric rows, 60 same-graph rows, 29 transitions, 180 stage timings.
results/run_manifest.json	Input/config hashes, algorithm-source hashes, Python, Java and package versions.
results/ai/	Actual AI prompts, provided input, output, coverage audit and comparison.
report/report_manifest.json	Hashes of files read by the report and generated PDF/figures; no recomputed partitions.

Recorded software: Python 3.12.14; matplotlib 3.10.8; numpy 2.3.5; reportlab 4.4.9; scikit-learn 1.8.0; scipy 1.17.0. The full Java toolchain identity and operating-system string are retained in run_manifest.json. Stable inputs, ordering and seeds make the procedure repeatable; environment changes can still affect floating-point details and wall time. [D11]

`Selection SHA-256: f0402f039117ca8f29499bcc64b0f848c7bbc0e0810260760ad1a4b6fe672c5d`
`Configuration SHA-256: c9cdafab5f35e5df2d1b9947a5b3fd0f3b5fca5cc51b60f26de1c64e2a8c6376`

The published repository packages raw artifacts and full per-release graph/result evidence in deterministic ZIP bundles under archives/versions. Unpacking restores the documented paths; summaries, source code, figures and guides remain directly readable. No source artifact is replaced by a derived summary.

Independent audit status: **passed**, 0 failures. A separate standard-library implementation recalculates exact-rational UE, lecture MQ and contingency-table ARI from stored evidence. It checks all 120 final partitions. This establishes internal consistency, not completeness of static analysis or intended-design accuracy. [D13]

Appendix A1 Complete quality results

Selected releases 1–10 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
10.0	A1	317	45	0.3290	0.0073	0.3217	65%	40
10.0	A2	311	45	0.3609	0.0069	0.3539	60%	49
10.0	A3	317	13	0.1317	0.0095	0.1221	61%	48
10.0	A4	311	13	0.1527	0.0114	0.1413	61%	54
11.0	A1	355	51	0.3522	0.0079	0.3442	65%	45
11.0	A2	346	51	0.3780	0.0073	0.3707	60%	43
11.0	A3	355	14	0.1472	0.0114	0.1357	64%	53
11.0	A4	346	14	0.1282	0.0080	0.1202	57%	40
12.0	A1	373	53	0.3955	0.0101	0.3853	66%	46
12.0	A2	365	52	0.4128	0.0084	0.4044	62%	59
12.0	A3	373	14	0.1235	0.0102	0.1133	71%	64
12.0	A4	365	14	0.1324	0.0075	0.1249	60%	50
13.0.1	A1	382	59	0.3786	0.0092	0.3694	70%	45
13.0.1	A2	374	58	0.3781	0.0084	0.3697	66%	58
13.0.1	A3	382	14	0.1077	0.0111	0.0966	70%	70
13.0.1	A4	374	14	0.1273	0.0083	0.1190	63%	60
15.0	A1	441	73	0.3955	0.0077	0.3878	70%	37
15.0	A2	430	74	0.3933	0.0059	0.3874	65%	40
15.0	A3	441	15	0.1211	0.0085	0.1126	69%	91
15.0	A4	430	15	0.1102	0.0074	0.1027	58%	81
16.0.1	A1	445	73	0.3888	0.0078	0.3810	70%	49
16.0.1	A2	435	76	0.3822	0.0058	0.3764	64%	51
16.0.1	A3	445	15	0.1080	0.0137	0.0944	70%	83
16.0.1	A4	435	15	0.1198	0.0068	0.1130	61%	72
18.0	A1	451	77	0.3949	0.0070	0.3879	69%	49
18.0	A2	441	79	0.3925	0.0049	0.3876	64%	39
18.0	A3	451	16	0.0863	0.0120	0.0743	75%	57
18.0	A4	441	16	0.1299	0.0120	0.1179	67%	64
21.0	A1	518	78	0.4129	0.0068	0.4061	69%	48
21.0	A2	502	78	0.4016	0.0051	0.3966	63%	42
21.0	A3	518	17	0.1102	0.0106	0.0996	71%	61
21.0	A4	502	17	0.1249	0.0101	0.1148	63%	91
23.0	A1	538	81	0.3973	0.0064	0.3909	70%	72
23.0	A2	521	80	0.3907	0.0048	0.3859	64%	75
23.0	A3	538	17	0.0809	0.0085	0.0723	71%	60
23.0	A4	521	17	0.1257	0.0068	0.1189	64%	82
23.2	A1	539	83	0.3873	0.0068	0.3805	70%	49
23.2	A2	522	82	0.3854	0.0050	0.3804	65%	75
23.2	A3	539	17	0.1112	0.0129	0.0983	73%	102
23.2	A4	522	17	0.1052	0.0077	0.0975	65%	82

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

Appendix A2 Complete quality results

Selected releases 11–20 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
23.5	A1	541	84	0.4054	0.0065	0.3989	70%	60
23.5	A2	525	82	0.3984	0.0047	0.3937	65%	38
23.5	A3	541	17	0.1226	0.0092	0.1135	71%	94
23.5	A4	525	17	0.1321	0.0068	0.1254	56%	123
24.0	A1	546	87	0.3906	0.0067	0.3839	71%	47
24.0	A2	529	85	0.3914	0.0047	0.3867	66%	40
24.0	A3	546	17	0.1058	0.0085	0.0973	73%	62
24.0	A4	529	17	0.1043	0.0061	0.0982	68%	77
25.1	A1	551	88	0.3977	0.0065	0.3912	71%	76
25.1	A2	534	87	0.4023	0.0044	0.3979	65%	59
25.1	A3	551	17	0.0877	0.0099	0.0778	71%	86
25.1	A4	534	17	0.1206	0.0089	0.1117	65%	84
27.0	A1	554	88	0.3871	0.0063	0.3808	71%	44
27.0	A2	536	87	0.3799	0.0045	0.3754	66%	42
27.0	A3	554	17	0.0910	0.0110	0.0800	71%	76
27.0	A4	536	17	0.1134	0.0056	0.1078	61%	88
27.1	A1	553	88	0.3912	0.0063	0.3849	71%	47
27.1	A2	535	88	0.3863	0.0046	0.3817	66%	45
27.1	A3	553	17	0.1125	0.0116	0.1010	71%	105
27.1	A4	535	17	0.1487	0.0069	0.1418	61%	100
28.2	A1	555	90	0.3846	0.0061	0.3785	71%	44
28.2	A2	536	89	0.3929	0.0046	0.3882	66%	42
28.2	A3	555	17	0.0887	0.0118	0.0769	74%	60
28.2	A4	536	17	0.1259	0.0047	0.1212	63%	84
30.0	A1	566	96	0.4069	0.0061	0.4007	72%	48
30.0	A2	547	96	0.4120	0.0051	0.4069	67%	46
30.0	A3	566	17	0.1041	0.0075	0.0966	70%	75
30.0	A4	547	17	0.1148	0.0080	0.1068	63%	96
30.1.1	A1	568	98	0.3994	0.0085	0.3909	73%	42
30.1.1	A2	548	96	0.4019	0.0052	0.3967	67%	44
30.1.1	A3	568	17	0.0834	0.0075	0.0760	71%	73
30.1.1	A4	548	17	0.1077	0.0089	0.0988	68%	101
31.1	A1	600	96	0.4109	0.0062	0.4047	70%	47
31.1	A2	578	96	0.4004	0.0055	0.3949	67%	70
31.1	A3	600	18	0.0940	0.0070	0.0870	66%	83
31.1	A4	578	18	0.0979	0.0082	0.0897	57%	46
32.0.1	A1	604	96	0.4152	0.0062	0.4090	70%	44
32.0.1	A2	582	96	0.4017	0.0057	0.3960	66%	65
32.0.1	A3	604	18	0.0993	0.0109	0.0884	71%	68
32.0.1	A4	582	18	0.1051	0.0060	0.0991	54%	71

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

Appendix A3 Complete quality results

Selected releases 21–30 of 30. Each row is the only final partition for that release and condition. Full precision and per-cluster/pair terms are stored in the result artifacts. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
32.1.2	A1	605	96	0.4197	0.0064	0.4133	70%	48
32.1.2	A2	583	96	0.4113	0.0051	0.4062	66%	67
32.1.2	A3	605	18	0.1054	0.0072	0.0982	64%	68
32.1.2	A4	583	18	0.1079	0.0090	0.0989	58%	57
33.0.0	A1	602	93	0.4015	0.0060	0.3955	70%	44
33.0.0	A2	579	92	0.3941	0.0055	0.3885	67%	60
33.0.0	A3	602	18	0.1147	0.0084	0.1064	64%	66
33.0.0	A4	579	18	0.1045	0.0053	0.0993	60%	82
33.2.0	A1	606	94	0.4062	0.0061	0.4001	70%	46
33.2.0	A2	583	95	0.3949	0.0053	0.3896	67%	56
33.2.0	A3	606	18	0.0822	0.0099	0.0723	69%	64
33.2.0	A4	583	18	0.0948	0.0070	0.0878	59%	84
33.3.1	A1	608	94	0.3961	0.0060	0.3902	70%	46
33.3.1	A2	584	92	0.4006	0.0047	0.3959	65%	68
33.3.1	A3	608	18	0.1011	0.0084	0.0927	64%	74
33.3.1	A4	584	18	0.1016	0.0068	0.0947	56%	75
33.4.1	A1	596	97	0.3675	0.0051	0.3625	70%	52
33.4.1	A2	573	96	0.3694	0.0042	0.3652	65%	62
33.4.1	A3	596	18	0.0932	0.0106	0.0826	73%	63
33.4.1	A4	573	18	0.1333	0.0077	0.1256	61%	113
33.4.3	A1	596	97	0.3675	0.0051	0.3625	70%	52
33.4.3	A2	573	96	0.3694	0.0042	0.3652	65%	62
33.4.3	A3	596	18	0.0932	0.0106	0.0826	73%	63
33.4.3	A4	573	18	0.1333	0.0077	0.1256	61%	113
33.4.6	A1	595	93	0.3853	0.0052	0.3802	70%	76
33.4.6	A2	572	93	0.3813	0.0039	0.3773	65%	104
33.4.6	A3	595	18	0.1001	0.0117	0.0884	73%	97
33.4.6	A4	572	18	0.1570	0.0097	0.1473	67%	101
33.4.8	A1	595	93	0.3819	0.0052	0.3767	70%	76
33.4.8	A2	572	93	0.3822	0.0040	0.3782	65%	89
33.4.8	A3	595	18	0.1001	0.0117	0.0884	73%	97
33.4.8	A4	572	18	0.1587	0.0099	0.1488	67%	101
33.7.0	A1	591	95	0.4078	0.0046	0.4032	70%	69
33.7.0	A2	569	98	0.3920	0.0040	0.3880	65%	97
33.7.0	A3	591	18	0.1108	0.0102	0.1006	75%	96
33.7.0	A4	569	18	0.1272	0.0080	0.1192	67%	93
33.7.2	A1	591	95	0.4078	0.0046	0.4032	70%	69
33.7.2	A2	569	98	0.3920	0.0040	0.3880	65%	97
33.7.2	A3	591	18	0.1108	0.0102	0.1006	75%	96
33.7.2	A4	569	18	0.1272	0.0080	0.1192	67%	93

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

References and source index

Supplied teaching material

[AS] EPL484, Fall 2026, Project 1 instructions, 5 PDF pages. Eligibility p.1; data pp.2–3; AI and report pp.3–5.

[TUT] Project 1 tutorial, 12 PDF pages. Deadline p.2; relation types p.6; ACDC pp.7–8; clustering pp.10–12.

[L12] Lectures 1–2: Introduction, 50 PDF pages. Evolution, maintenance and S/P/E classification pp.15–33; Lehman's laws and limits pp.35–48.

[L3] Lecture 3: Requirements, Part 1, 38 PDF pages. Measurable definitions/automation pp.9–19; relationship and traceability models pp.27–33.

[L4] Lecture 4: Requirements, Part 2, 33 PDF pages. Traceability and its limits pp.11–15; feedback evidence pp.22–31.

[L5] Lecture 5: Architecture Evolution, 34 PDF pages. PDF p.9 = printed slide10; after the missing slide6, PDF and printed numbers differ by one.

[L67] Lectures 6–7: Architecture Reconstruction, 50 PDF pages. MQ equations and worked example pp.15–17; ACDC pp.19–24; older Noise method pp.46–47.

Research and primary documentation

[UE] Yano and Matsuo (2020), Moderate Detection and Removal of Omnipresent Modules in Software Clustering, ICSME, pp.662–666. Equations1–3: p.663; procedure: p.664. https://doi.org/10.1109/ICSME46990.2020.00068

[NOISE] Constantinou, Kakarontzas and Stamelos (2015), JSS107, pp.142–157. Older significance-based method, distinguished from this UE replacement. https://doi.org/10.1016/j.jss.2015.05.065

[ACDC] Original ACDC author archive, preserved and SHA-256 pinned in third_party/acdc.
https://www.eecs.yorku.ca/~bil/downloads/acdc.tar

[SK] scikit-learn 1.8 KMeans API and normalize API; actual installed version is recorded by the run.
https://scikit-learn.org/1.8/modules/generated/sklearn.cluster.KMeans.html

[JVMS] Java Virtual Machine Specification21, §5.4.3.2–5.4.3.4: field and method resolution.
https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-5.html

[G1] Guava source repository and project overview. https://github.com/google/guava

[G2] Guava ReleaseHistory, with linked historical release notes. https://github.com/google/guava/wiki/ReleaseHistory

Repository evidence cited in this report

[D1] results/sizes.csv; metrics.csv; versions/*/*_metrics.json

[D2] docs/omnipresent.md; src/evolution/omnipresent.py; results/versions/*/ue.json

[D3] results/same_graph_comparisons.csv; versions/*/*_projected_to_retained_metrics.json

[D4] docs/project_selection.md; config/versions.json; data/provenance/tag_inventory.json; project_eligibility.json

[D5] docs/data_acquisition.md; scripts/fetch_data.py; data/provenance/acquisition.json and source/Git audits

[D6] docs/modeling_decisions.md; extractor source; data/graphs/*/provenance.json and diagnostics

[D7] docs/course_traceability.md; src/evolution/clustering.py; config/analysis.json; A1–A4 run metadata

[D8] results/pilot/pilot.json and associated pilot assignments

[D9] src/evolution/metrics.py; tests; results/versions/*/*_quality/

[D10] results/evolution.csv; canonical class and edge manifests

[D11] results/runtimes.csv; run_manifest.json; performance/ue_benchmark.json

[D12] results/ai/experiment.json and its exact prompt/input/output/quality artifacts

[D13] scripts/validate_results.py; results/validation.json, independent artifact audit

[D14] docs/ue_case_study.md; results/performance/case_checks.json, exact-fraction source-backed UE cases

[D15] scripts/audit_change_points.py; results/change_points.json, source-backed annotation and graph-difference audit